

Reverse a Doubly Linked List

Final Study Notebook — Transcript-Based

This version follows the requested notebook order: **Introduction** → **Problem Statement** → **Required Concepts** → **Approaches** → **Pseudocode** → **Complete Java Code** → **Visual Representation** → **Time & Space Complexity** → **Conclusion**. The content is grounded in the supplied transcript. ■filecite■turn1file0■L19-L49■

1. Introduction

A doubly linked list contains nodes connected in two directions. Each node has a value, a **next** link and a **previous/back** link. Because both directions are available, the list can be traversed forward and backward. ■filecite■turn1file0■L55-L93■

NULL ← [1] ■ [2] ■ [3] ■ [4] → NULL
 HEAD

The topic of the video is **reversing this doubly linked list**. The required result is the same nodes in reverse order, with the links pointing in the correct reversed directions.

2. Problem Statement

You are given the **head** of a doubly linked list. Reverse the doubly linked list and return the **new head**. This is the exact task introduced in the transcript. ■filecite■turn1file0■L19-L49■

Input	Required Output
NULL	NULL
[10]	[10]
NULL ← [10] ■ [20] ■ [30] → NULL	NULL ← [30] ■ [20] ■ [10] → NULL

3. Small Concepts Required

Only a few concepts are needed before solving the problem:

Concept	Meaning
data	The value stored inside the node.
next	Reference to the next node in the forward direction.
prev	Reference to the previous node in the backward direction.
head	Reference to the first node of the current list.
new head	After reversal, the old tail becomes the first node.

For the optimized solution, the important idea is: **do not move the data; reverse the links**. The transcript first explores data reversal, then explicitly moves to reversing links for a one-pass solution. ■filecite■turn1file0■L599-L691■

4. Approaches Explained in the Video

The transcript develops two approaches.

Approach	Main idea	Passes	Extra space
1. Extreme / Brute Force	Use a stack to reverse node data values.	2	O(n)
2. Optimized	Swap each node's prev and next links.	1	O(1)

The first method is introduced as the 'extreme naive' idea. The second method is the optimization the transcript develops after the interviewer asks for a one-pass solution. ■filecite■turn1file0■L99-L119■
■filecite■turn1file0■L577-L597■

5. Approach 1 — Stack + Data Reversal

The idea is simple: collect the node values in a stack. Because a stack is LIFO, popping the values returns them in reverse order. Then traverse the linked list again and replace each node's data. ■filecite■turn1file0■L153-L165■

```
Pass 1:
temp = head
while temp != null:
    stack.push(temp.data)
    temp = temp.next
```

```
Pass 2:
temp = head
while temp != null:
    temp.data = stack.top()
    stack.pop()
    temp = temp.next
```

The transcript describes this as reversing the list **only in terms of data**; the actual links are not changed. ■filecite■turn1file0■L315-L325■

6. Complete Java Code — Approach 1

```
import java.util.Stack;

class Solution {
    public ListNode reverseDLLBruteForce(ListNode head) {

        // Store all node values in LIFO order.
        Stack<Integer> stack = new Stack<>();

        ListNode temp = head;

        // First traversal: push data.
        while (temp != null) {
            stack.push(temp.data);
            temp = temp.next;
        }

        // Second traversal: replace data from the stack.
        temp = head;

        while (temp != null) {
            temp.data = stack.pop();
            temp = temp.next;
        }

        // The links are unchanged; only the data is reversed.
        return head;
    }
}
```

The code directly implements the two-pass pseudocode described in the transcript. ■filecite■turn1file0■L327-L487■

7. Approach 2 — One-Pass Link Swapping

The optimized idea is to reverse the **connections** of each node instead of changing its data. The transcript first isolates one node and observes that the node's next and previous links simply need to exchange places.

■filecite■turn1file0■L693-L843■

Before:
old prev ← [CURRENT] → old next

After:
old next ← [CURRENT] → old prev

Because changing current.prev destroys the old previous reference, save it first. The transcript relates this to the normal temporary-variable method used when swapping two variables. ■filecite■turn1file0■L843-L915■

Pseudocode

```
if head == null OR head.next == null:
    return head

previous = null
current = head

while current != null:
    previous = current.prev
    current.prev = current.next
    current.next = previous
    current = current.prev

return previous
```

The transcript's final implementation follows this same structure: base case, previous/current initialization, four operations inside the loop, then return the final previous pointer as the new head. ■filecite■turn1file0■L1557-L1671■

8. Complete Java Code — Approach 2

```
class Solution {
    public ListNode reverseDLL(ListNode head) {

        // Empty list or one-node list is already reversed.
        if (head == null || head.next == null) {
            return head;
        }

        ListNode previous = null;
        ListNode current = head;

        while (current != null) {

            // Save the old previous link before overwriting it.
            previous = current.prev;

            // Swap the directions of the two links.
            current.prev = current.next;
            current.next = previous;

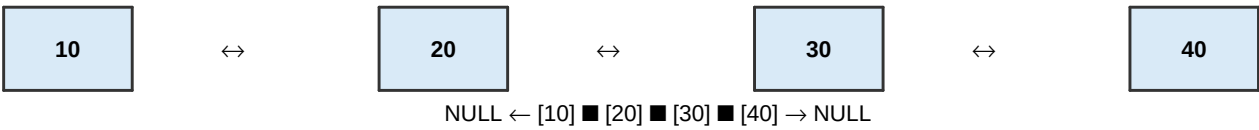
            // After the swap, current.prev points to
            // the next node in the ORIGINAL list.
            current = current.prev;
        }

        // The old tail is now the new head.
        return previous;
    }
}
```

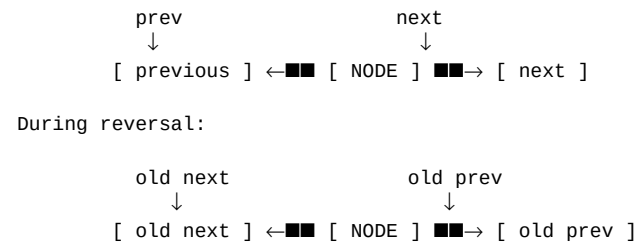
9. Visual Representation — Final Structure

This section is intentionally **not a step-by-step dry run**. It shows the kind of static visual representation used in algorithm documentation: the original structure, the pointer relationship, and the final structure after the algorithm.

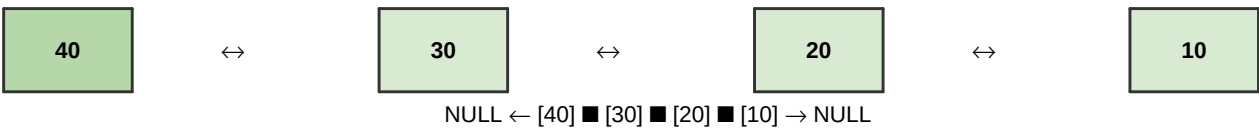
9.1 Original DLL



9.2 What each node's links represent



9.3 Reversal — final documented structure



40 is the new head. The old tail becomes the new head because the directions of all links have been reversed. The transcript explicitly identifies the final saved previous/back pointer as the new head.

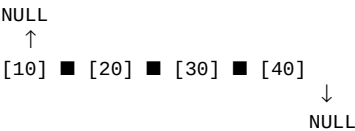
filecite turn1file0 L1443-L1527

9.4 Pointer-change summary

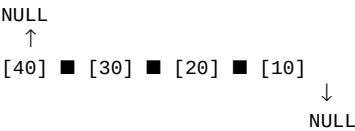
Original relationship	After reversal
current.prev → previous node	current.prev → next node
current.next → next node	current.next → previous node
old head	becomes old tail
old tail	becomes new head

9.5 Algorithm-style visual

ORIGINAL



REVERSED LINKS



NEW HEAD = 40

10. Time and Space Complexity

Approach	Time	Auxiliary Space	Why
Stack + data reversal	$O(n)$	$O(n)$	Two full traversals and a stack containing n values
Link swapping	$O(n)$	$O(1)$	One traversal; only a constant number of pointer v

The transcript explicitly states that the brute-force method is $O(n)$ time and $O(n)$ space, while the single-pass link-swapping method is $O(n)$ time and $O(1)$ space. ■filecite■turn1file0■L499-L575■
■filecite■turn1file0■L1697-L1749■

Why is two passes still $O(n)$? $O(n) + O(n) = O(2n)$, and constants are ignored in Big-O, so the final time complexity is $O(n)$.

11. Conclusion

The video first solves the problem using a stack by reversing node data. That approach is straightforward but uses $O(n)$ auxiliary space and requires two traversals. The optimized approach observes one node, swaps its previous and next links, repeats this for every node in a single traversal, and returns the old tail as the new head. ■filecite■turn1file0■L577-L597■ ■filecite■turn1file0■L1697-L1735■

Final memory line: For a doubly linked list reversal, swap **prev** and **next** for every node. Save the old **prev** first, move using the swapped **prev**, and return the old tail as the new head.